Report on DA4005 Programming Exercise 1

A Turing Machine for Unary Cubes

Onyinyechi Sonia Bäckström

Department of Mathematics
Stockholm University

September 2026

1 Task

The task is to construct a Turing machine that accepts the unary word a^n if and only if

$$n = k^3$$

for some integer $k \geq 0$. Thus, the lengths $0, 1, 8, 27, 64, 125, \ldots$ are accepted, while non-cube lengths such as $2, 7, 9, 26, 28, 63, 65, 100, 124, 126$ are rejected.

The program receives exactly one command-line argument containing zero(input: `""`) or more copies of the character `a`. The empty argument represents zero. Any malformed input, missing argument, extra argument, or character other than `a` is rejected.

2 Implementation

I implemented the required unary-cube recognizer as a Turing machine in Python. The program takes one command-line argument, puts the input on the supplied tape, and then performs the calculation using the tapes and a fixed set of control operations.

I used four tapes: one input tape and three work tapes. The work tapes store the remainder, the current cube difference, and k. I chose this representation because the separate work tapes make the unary values easier to manipulate while keeping all input-dependent information on explicit tapes. The number of tapes is fixed and does not depend on the input length.

- **Tape 1: input tape.** Contains the input word in unary. Each input symbol a is represented by `ONE`.
- **Tape 2: remainder tape.** Stores the number of input symbols that have not yet been consumed.
- **Tape 3: delta tape.** Stores the current difference between two consecutive cubes.
- **Tape 4: k tape.** Stores the current value of k in unary.

The work tapes are initialized with a `START` marker at the beginning. Their values are stored as `ONE` symbols to the right of `START`. The input symbols are also represented by `ONE`.

The program expects one command-line argument. An empty command-line argument represents unary zero. Inputs containing characters other than a are rejected. When testing zero in the Anaconda Prompt, the input is written as `""`. The `""` is shell syntax for an empty argument, which gives the program an empty argument. The machine accepts because the length is 0 and

$$0 = 0^3.$$

I did not use input-dependent Python lists, dictionaries, recursion, or numeric counters for the machine's calculation. Values that can grow, such as the remainder, delta, and k, are stored on the tapes.

If the input is a cube, the program calls `Tape.accept()` and prints `[accept]`. Otherwise, it calls `Tape.reject()` and prints `[reject]`.

2.1 Tape Alphabet and Layout

The tape alphabet is

$$\Gamma = \{\texttt{BLANK}, \texttt{ONE}, \texttt{START}, \texttt{MARK}\}.$$

Therefore,

$$|\Gamma| = 4.$$

The symbols have the following meanings:

- `BLANK`: the blank tape cell and the end of a written unary value.
- `ONE`: represents one unit of a unary value.

- `START`: marks the beginning of a work-tape value.
- `MARK`: marks a `ONE` on the remainder tape that has already been consumed.

Each work tape has the form

`START ONE ONE ...ONE BLANK.`

For example, the value 3 is represented as

`START ONE ONE ONE BLANK.`

Zero is represented by

`START BLANK.`

The input tape contains the unary input directly as `ONE` symbols. The supplied tape implementation is responsible for the input-dependent tape storage. The machine computation uses no input-dependent Python list, dictionary, string, recursion, or numeric counter.

2.2 Tape Operations and Head Positions

The important routines have fixed starting and ending head positions.

- `rewind()` starts with the head at any position on a work tape. It moves left until it reaches `START`, then moves one cell right. It therefore ends on the first cell of the unary value.
- `append_one()` starts at the beginning of a unary value and moves right over its `ONE` symbols. It finishes on the first blank cell, where it writes a new `ONE`.
- `increment()` first rewinds the tape and then appends one `ONE`. It finishes on the newly appended symbol.
- `copy_input()` starts with the input head at the first input symbol and copies each `ONE` to the remainder tape. It finishes when the input head reaches the blank cell after the input.
- `subtract()` rewinds both the source and target tapes before beginning. It consumes one unused target `ONE` for every source `ONE`. If the target does not contain enough unused symbols, the routine reaches `BLANK` and rejects.
- `add()` rewinds the source and target tapes and adds the unary value in the source to the target.
- `add_six_times()` performs the addition operation six times and is used to calculate the next cube difference.

The routines operate only on the symbols allowed by the documented tape layout. If a required target symbol is unavailable during subtraction, the machine rejects rather than accepting the input.

3 How the Algorithm Works

The machine tests whether the input length is a perfect cube by subtracting the differences between consecutive cubes.

The differences are

$$1^3 - 0^3 = 1,$$
$$2^3 - 1^3 = 7,$$
$$3^3 - 2^3 = 19,$$
$$4^3 - 3^3 = 37.$$

Thus the sequence is

$$1, 7, 19, 37, 61, \ldots$$

Let

$$d_k = k^3 - (k-1)^3.$$

The next difference can be calculated without multiplication by using

$$d_{k+1} = d_k + 6k.$$

The machine starts with

$$k = 1, \qquad d_1 = 1.$$

It copies the input to the remainder tape and then repeatedly subtracts the current cube difference from the remainder.

If the remainder becomes zero, the machine accepts. If the next difference is larger than the remaining number of symbols, the machine rejects. Otherwise, the machine calculates the next difference and increments k.

4 Correctness Argument

4.1 Loop Invariant

At the beginning of each iteration, let the value stored on the k-tape be k and let the value stored on the remainder tape be r. The following invariant holds:

$$r = n - (k-1)^3,$$

where n is the original input length, and the value stored on the delta tape is

$$d_k = k^3 - (k-1)^3.$$

Therefore, the amount already removed from the remainder is exactly

$$(k-1)^3.$$

The next subtraction attempts to remove

$$d_k = k^3 - (k-1)^3,$$

which would make the total number of removed symbols equal to k^3.

4.2 Acceptance

If subtracting d_k makes the remainder zero, then

$$n - (k-1)^3 = d_k.$$

Since

$$d_k = k^3 - (k-1)^3,$$

it follows that

$$n = k^3.$$

Therefore the input length is a perfect cube and the machine accepts.

4.3 Rejection

Suppose the next difference cannot be completely subtracted from the remainder. Then

$$n < k^3,$$

while the invariant gives

$$n \geq (k-1)^3.$$

Thus

$$(k-1)^3 \leq n < k^3,$$

so the input length lies strictly between two consecutive cubes. Therefore it cannot be a perfect cube and the machine correctly rejects.

4.4 Why Every Cube Is Reached

The value of k starts at 1 and is increased by one after every successful subtraction. The delta value is updated according to

$$d_{k+1} = d_k + 6k.$$

Since

$$d_k = k^3 - (k-1)^3,$$

the machine considers the consecutive cube boundaries

$$1^3, 2^3, 3^3, 4^3, \ldots$$

Therefore, for every finite input length n, the machine eventually either reaches $n = k^3$ and accepts, or reaches the first cube larger than n and rejects.

5 Halting Argument

5.1 Termination of Tape Routines

Each tape routine operates on a finite portion of a tape.

The `rewind()` routine moves left until it reaches the fixed `START` marker and therefore terminates. The `append_one()` routine moves right through a finite unary block until it reaches the first blank cell. The `increment()` routine therefore also terminates.

The `copy_input()` routine processes each input symbol once and terminates when it reaches the end of the input. The `subtract()` routine processes one source `ONE` at each iteration. It either processes all source symbols or reaches a blank target cell and rejects. Therefore it terminates.

The `add()` routine processes each `ONE` in its finite source value once and therefore terminates. Since `add_six_times()` performs exactly six calls to `add()`, it also terminates.

5.2 Termination of the Main Loop

The outer loop also terminates for every valid finite input.

The cube differences are positive, so the number of symbols removed from the remainder strictly increases after every successful iteration. If the input length is a perfect cube, the remainder eventually becomes zero and the machine accepts.

If the input length is not a perfect cube, the machine eventually reaches the first cube greater than the input length. The corresponding cube difference is then larger than the remaining number of symbols, so `subtract()` rejects.

Consequently, the main loop cannot continue indefinitely, and the machine halts on every valid input.

6 Memory Restriction

All input-dependent values used by the computation are stored on the explicit tapes. In particular, the remainder, the current cube difference, and k are represented in unary on the work tapes.

The program does not compute the input length as a Python integer and does not use host-language arithmetic to test whether the input length is a cube. Python variables are used only for fixed control and references to the fixed set of tapes.

The number of tapes and the tape alphabet are fixed independently of the input length. The command-line argument is inspected only during tape initialization.

7 Testing

The following test cases were used. The accepted lengths are perfect cubes, while the rejected lengths lie between consecutive cubes.

Length	Input	Expected	Observed
0	`""`	accept	accept
1	`a`	accept	accept
2	`aa`	reject	reject
7	`aaaaaaa`	reject	reject
8	`aaaaaaaa`	accept	accept
9	`aaaaaaaaa`	reject	reject
26	a^{26}	reject	reject
27	a^{27}	accept	accept
28	a^{28}	reject	reject
63	a^{63}	reject	reject
64	a^{64}	accept	accept
65	a^{65}	reject	reject
100	a^{100}	reject	reject
124	a^{124}	reject	reject
125	a^{125}	accept	accept
126	a^{126}	reject	reject

Table 1: Test cases for unary cube recognition.

In particular,

$$64 = 4^3, \qquad 125 = 5^3,$$

whereas

$$64 < 100 < 125.$$

Therefore, 100 is not a cube and must be rejected.

8 Instructions on How to Run

The files `accept_cube.py` and `tape.py` must be in the same folder.

The program is run with

```
python accept_cube.py <input>
```

For example, an input of length 8 is run using

```
python accept_cube.py aaaaaaaa
```

which produces

```
[accept]
```

An input of length 7 is run using

```
python accept_cube.py aaaaaaa
```

which produces

```
[reject]
```

The empty argument represents zero is run using

```
python accept_cube.py ""
```

which produces

```
[accept]
```

References

[Alexandersson(2026)] Per Alexandersson. *DA4005 – Lecture 1 Organization and Turing machines*. Department of Mathematics, Stockholm University, 2026.

[Sipser(2013)] Michael Sipser. *Introduction to the Theory of Computation*. 3rd edition, Cengage Learning, 2013.